Generic Classes and Methods

CS102 – Introduction to Computer Science

Topics

- Learning Outcome
- The Scenario
- Why Generic Programming
- Generic Methods
- Generic Methods: Implementation and Compile-Time Translation
- Generic Methods: With return Type
- Generic Classes
- Programs

Learning Outcome

- To understand the objective of generic programming
- To be able to implement generic classes and methods

The Scenario

- Imagine that you've just finished writing a wonderful Sorting class that works perfectly for Integer numbers.
- But what about a sorting to hold
  - … Strings?
  - … or Double?
  - … or Characters?
- **Notice that in the process of sorting.**
  - We've given too much information to the class (supplier).
  - The algorithm (client) should specify what to hold.
  - The sorting shouldn't care about what it holds.
  - The sorting should simply to sort whatever items it holds like Integers String, Characters Double or Records etc.,
- To achieve the above said Scenario, we use

Generic Classes and Methods

Why Generic Programming

- Generic methods and generic classes (and interfaces) enable you to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.
- Generics also provide compile-time type safety that allows you to catch invalid types at compile time.
- **Java generics**
  - lets you write code that is safer and easier to read
  - is especially useful for general data structures, such as ArrayList
  - Starting with version 5.0, Java allows class definitions with parameters for types.
  - All generic class and method declarations have a type-parameter section delimited by angle brackets (< and >).

generic programming = programming with classes and methods parameterized with types

Generic Methods

- Overloaded methods are often used to perform similar operations on different types of data.

Program – 1 :

```
public class OverLoadedMethods {
      public static void main(String[] args) {
            Integer[] integerArray = { 3, 5, 1, 6, 2, 4, 7 };
            Double[] doubleArray = { 2.2, 4.4, 6.6, 3.3, 7.7, 5.5, 1.1 };
            Character[] characterArray = { 'G', 'E', 'N', 'E', 'R', 'I', 'C'};
            System.out.println( "Array integerArray contains:" );
            printArray( integerArray );
            System.out.println( "\nArray doubleArray contains:" );
            printArray( doubleArray );
            System.out.println( "\nArray characterArray contains:" );
            printArray( characterArray );
      }
      public static void printArray(Integer[] inputArray){
            for ( Integer element : inputArray )
                  System.out.printf( "%s ", element );
            System.out.println();
      }
      public static void printArray(Double[] inputArray){
            for ( Double element : inputArray )
                  System.out.printf( "%s ", element );
            System.out.println();
      }
      public static void printArray(Character[] inputArray){
            for ( Character element : inputArray )
                  System.out.printf( "%s ", element );
            System.out.println();
      }
}
```

Output :

```
Array integerArray contains:
3 5 1 6 2 4 7

Array doubleArray contains:
2.2 4.4 6.6 3.3 7.7 5.5 1.1

Array characterArray contains:
G E N E R I C
```

This method is invoked to print the integer values

This method is invoked to print the Double values

This method is invoked to print the Character values

Generic Methods

- If the operations performed by several overloaded methods are identical for each argument type, the overloaded methods can be more compactly and conveniently coded using a generic- method.
- You can write a single generic method declaration that can be called with arguments of different types.
- Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.
- By adding a generic method which prints any type of data from the array.
- By just adding the given block code and removing all the overloaded methods to the above program.

```
public static <T>  void printArray(T[] inputArray){
    for ( T element : inputArray )
        System.out.printf( "%s ", element );
    System.out.println();
}
```

Generic Methods: Implementation and Compile-Time Translation

Program – 2 : Demonstration of generic method

```
public class GenericMethodTest {
     public static void main(String[] args) {
          Integer[] integerArray = { 3, 5, 1, 6, 2, 4, 7 };
          Double[] doubleArray = { 2.2, 4.4, 6.6, 3.3, 7.7, 5.5, 1.1 };
          Character[] characterArray = { 'G', 'E', 'N', 'E', 'R', 'I', 'C'};
          System.out.println( "Array integerArray contains:" );
          printArray( integerArray );
          System.out.println( "\nArray doubleArray contains:" );
          printArray( doubleArray );
          System.out.println( "\nArray characterArray contains:" );
          printArray( characterArray );
     }
     public static <T>  void printArray(T[] inputArray){
          for ( T element : inputArray )
               System.out.printf( "%s ", element );
          System.out.println();
     }
}
```

Output :

```
Array integerArray contains:
3 5 1 6 2 4 7

Array doubleArray contains:
2.2 4.4 6.6 3.3 7.7 5.5 1.1

Array characterArray contains:
G E N E R I C
```

Generic Methods: With return Type

- Generic method maximum determines and return a the largest of its three arguments of the same type.
- The relational operator > cannot be used with reference types, but it's possible to compare two objects of the same class if that class implements the generic interface Comparable<T> (package java.lang).
- Like generic classes, generic interfaces enable you to specify, with a single interface declaration, a set of related types.
- Comparable<T> objects have a compareTo method.
- The method must return zero if the objects are equal, a negative integer if object1 is less than object2 or a positive integer if object1 is greater than object2.
- A benefit of implementing interface Comparable<T> is that Comparable<T> objects can be used with the sorting and searching methods of class Collections (package java.util).

Generic Methods: With return Type

Program – 3 : Demonstration of generic method with return type

```
public class MaximumTest {
     public static void main(String[] args) {
          System.out.printf( "Maximum of 3, 4 and 5 is %d\n",
          maximum( 3, 4, 5 ));
          System.out.printf( "Maximum of 6.6, 8.8 and 7.7 is %.1f\n",
          maximum(6.6, 8.8, 7.7));
          System.out.printf( "Maximum of pear, apple and orange is %s\n",
          maximum("pear", "apple", "orange"));
     }
     public static < T extends Comparable< T > > T maximum( T x, T y, T z ){
          T max = x;
          if ( y.compareTo( max ) > 0 )
               max = y;
          if ( z.compareTo( max ) > 0 )
               max = z;
          return max;
     }
}
```

Output :

```
Maximum of 3, 4 and 5 is 5
Maximum of 6.6, 8.8 and 7.7 is 8.8
Maximum of pear, apple and orange is pear
```

Generic Classes

- Generic classes provide a means for describing the concept r class in a type-independent manner.
- These classes are known as parameterized classes or parameterized types because they accept one or more type parameters.
- Generic class declared with one or more type parameters
- A type parameter for **ArrayList** denotes the element type

```
public class ArrayList<E>{
   public ArrayList() {
   . . .
   }
   public void add(E element) {
   . . .
   }
   . . .
}
```

- Cannot use a primitive type as a type variable

```
ArrayList<double> //Wrong
```

- Use corresponding wrapper class instead

```
ArrayList<Double>
```

Generic Classes

Program – 4 : Demonstration of generic class

```
public class BoxGeneric<T> {
     private T t;
          public void setT(T t){
          this.t = t;
     }
     public T getT(){
          return this.t;
     }
}
public class BoxGenerixTest {
     public static void main(String[] args) {
          BoxGeneric<Integer> bgi = new BoxGeneric<Integer>();
          bgi.setT(10);
          System.out.println("The Integer value = " + bgi.getT());
          BoxGeneric<String> bgii = new BoxGeneric<String>();
          bgii.setT("Generic");
          System.out.print("The string value = " + bgii.getT());
     }
}
```

Output :

```
The Integer value = 10
The string value = Generic
```